OPERATING SYSTEMS AND SYSTEMS PROGRAMMING · 25CS2104E

Denying Writes to Executables

Linux-based executable file protection system

TEAM 06 | SECTION 12

2026–27 · TERM I

Hrushika Habeeb · 2520090054

Chakrika Panakala · 2520090042

Saranya Aluru · 2520080012

FACULTY · MR. M. RAGHUPATHI

Abstract

Executable files are critical system resources because they contain the instructions used to run programs.

If an unauthorized user or process can modify an executable, malicious code may be introduced without being immediately detected.

PREVENT

Restrict unauthorized write and modification operations.

MONITOR

Observe suspicious access and file-system activity.

VERIFY

Use SHA-256 hashes to check file integrity.

The project combines prevention, monitoring and integrity verification while preserving legitimate execution.

Problem Statement

What can go wrong?

- An unauthorized user or process gains write access.
- Original executable code can be altered or replaced.
- Malicious changes may remain unnoticed.
- Compromise can lead to data theft, unauthorized access or privilege escalation.

THE CORE CHALLENGE

Prevent unauthorized changes without stopping legitimate program execution.

Protect the file. Keep it usable.

Proposed Solution

A Linux-based mechanism to register selected executables and protect them from unauthorized modification.

01 REGISTER

Identify the executable and store its file details.

02 PROTECT

Apply Linux access controls to restrict unauthorized writes.

03 MONITOR

Watch for write, delete, rename and replacement attempts.

04 VERIFY

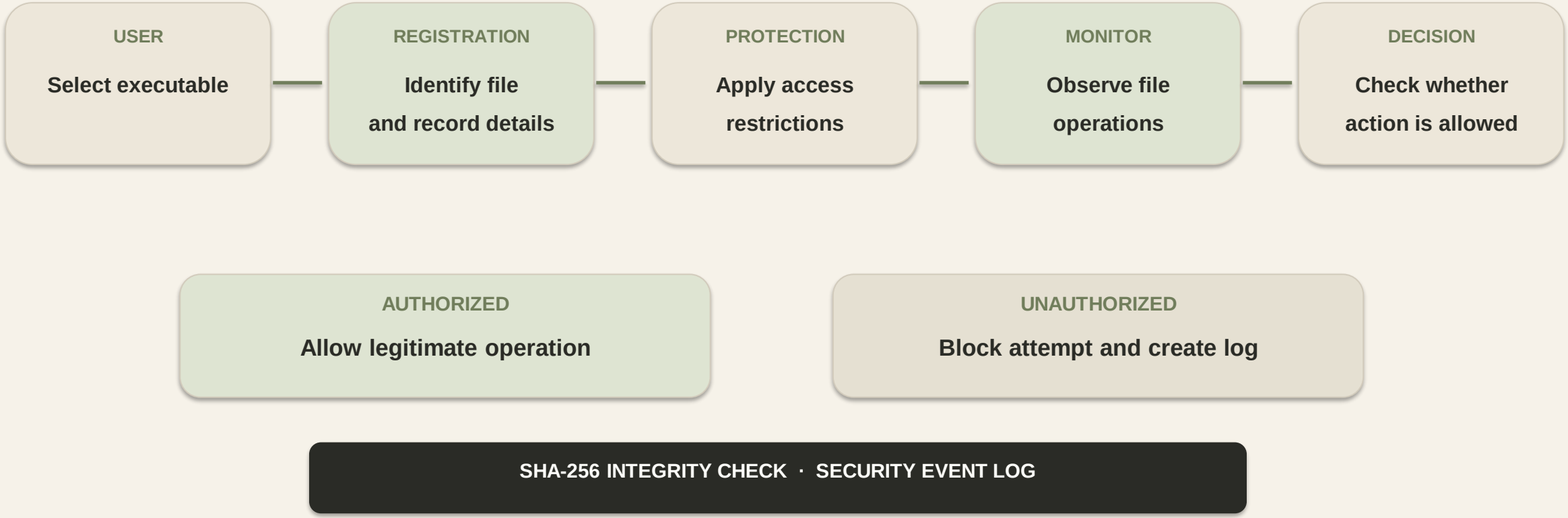
Compare the current SHA-256 hash with the stored reference.

05 RECORD

Log suspicious activity with process, user, operation and timestamp.

System Architecture

High-level workflow of the proposed protection system



Tools & Technologies Used

Linux (Ubuntu)

Operating system platform for implementation and testing.

C Programming

Core system-level protection and monitoring components.

GCC Compiler

Compile and build the C programs.

Linux Terminal / Bash

Run commands, test permissions and simulate write attempts.

VS Code

Write, edit and manage source code.

Git & GitHub

Version control and team collaboration.

OSSP Implementation

The project applies core Linux and system-programming mechanisms.

API / MECHANISM	ROLE IN THE PROJECT
chmod()	Change file permissions to restrict write access.
open()	Open files with a selected access mode.
stat()	Retrieve file details such as type, ownership and permissions.
write()	Handle write operations and demonstrate blocked modifications.
inotify	Monitor file-system events such as modification, deletion and rename.
SHA-256	Generate a cryptographic hash as an integrity reference.

Future Scope

Possible enhancements beyond the current project scope

CENTRALIZED MONITORING

Extend logs to a single dashboard for easier review.

ALERTS

Notify administrators when suspicious activity is detected.

POLICY CONTROL

Support different protection levels for different users.

SCHEDULED VERIFICATION

Run periodic integrity checks on protected executables.

Conclusion

Denying Writes to Executables focuses on protecting critical program files from unauthorized modification.

By combining Linux file permissions, system calls, file monitoring and SHA-256 integrity verification, the proposed system aims to prevent tampering while allowing legitimate execution.

01 **PREVENT**

02 **MONITOR**

03 **VERIFY**

Thank You.

Questions & Discussion

DENYING WRITES TO EXECUTABLES

TEAM 06 · SECTION 12